



SECP3623 DATABASE PROGRAMMING

TOPIC 10 – Advanced Operations in MongoDB

LECTURE LEARNING OUTCOME

By the end of this lecture, students should be able to:

- Apply indexing techniques in MongoDB to optimise query performance.
- Perform sorting operations on MongoDB query results.
- Use aggregation pipelines for data analysis.
- Execute advanced queries using MongoDB operators.

Create Database & Collection

- Database created: `shopdb`
- Collection created: `orders`

You can now proceed with:

- Insert documents
- Sorting
- Indexing
- Aggregation
- Advanced queries

Create Database

Database Name

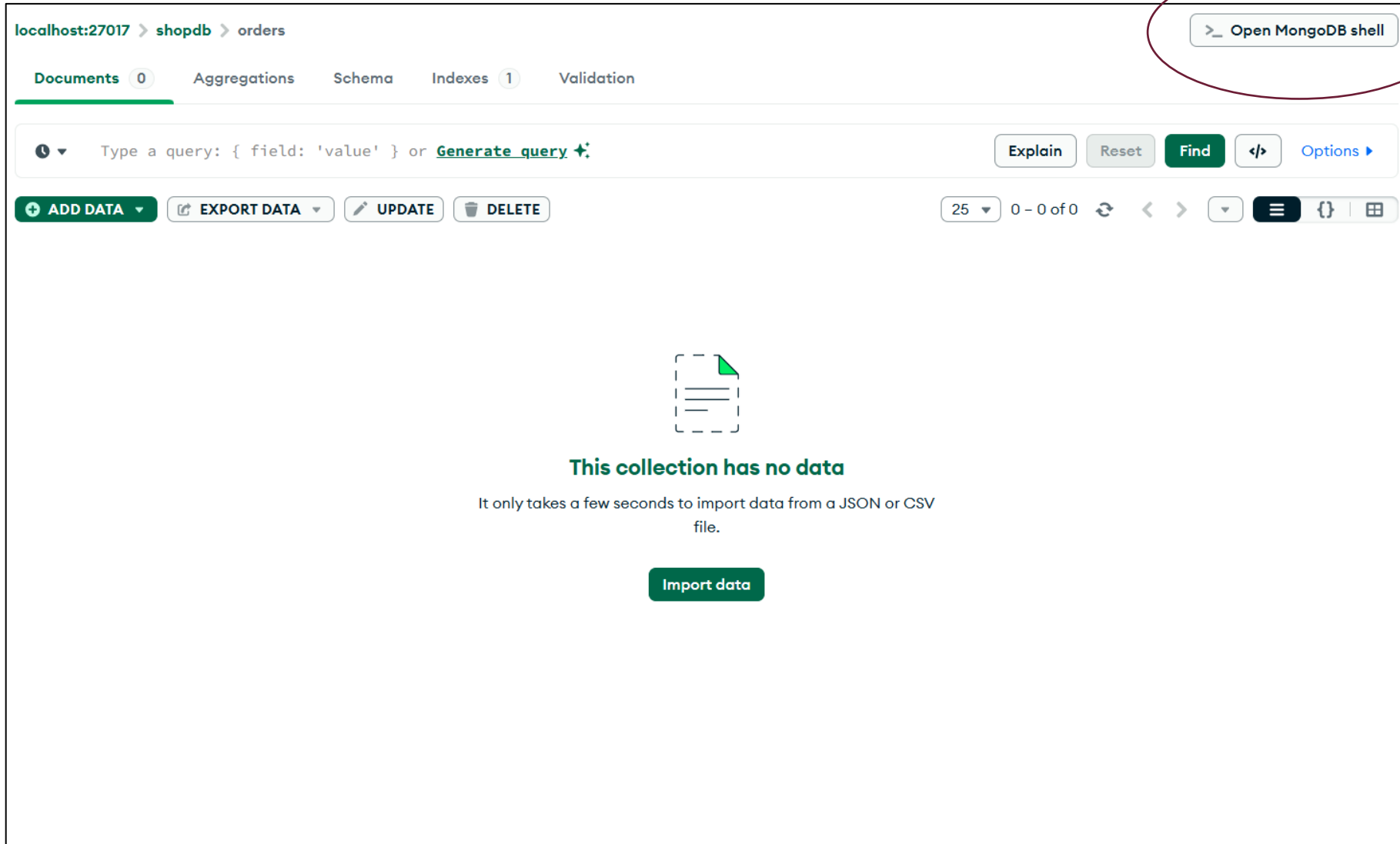
Collection Name

☐ **Time-Series**
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ **Additional preferences** (e.g. Custom collation, Clustered collections)

CancelCreate Database

Create Database & Collection



localhost:27017 > shopdb > orders

Documents 0 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ✨

Explain Reset Find </> Options ▸

+ ADD DATA ▾ EXPORT DATA ▾ UPDATE DELETE

25 0 - 0 of 0 ↺ < > ▾ ≡ {} ▢



This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Import data

>_ Open MongoDB shell

Insert Data

>_MONGOSH

```
<
> use shopdb
< switched to db shopdb
> db.orders.insertMany([
  { orderId: 1001, customer: "Ali",   city: "KL",      status: "PAID",    amount: 120, items: [{sku:"A1", qty:2}], orderDate: ISODate("2025-12-01")
  { orderId: 1002, customer: "Siti",  city: "JB",      status: "PENDING", amount: 80,  items: [{sku:"A1", qty:1}], orderDate: ISODate("2025-12-02")
  { orderId: 1003, customer: "Kumar",  city: "Penang",   status: "PAID",    amount: 300, items: [{sku:"C3", qty:3}], orderDate: ISODate("2025-12-02")
  { orderId: 1004, customer: "Mei",    city: "KL",      status: "FAILED",   amount: 50,  items: [{sku:"B2", qty:1}], orderDate: ISODate("2025-12-03")
  { orderId: 1005, customer: "Ali",    city: "KL",      status: "PAID",    amount: 200, items: [{sku:"D4", qty:2}], orderDate: ISODate("2025-12-04")
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('695d535d4b98f32ee2f091e6'),
    '1': ObjectId('695d535d4b98f32ee2f091e7'),
    '2': ObjectId('695d535d4b98f32ee2f091e8'),
    '3': ObjectId('695d535d4b98f32ee2f091e9'),
    '4': ObjectId('695d535d4b98f32ee2f091ea')
  }
}
```

Verify Data

```
>_MONGOSH
```

```
> db.orders.find()
```

```
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
{
  _id: ObjectId('695d535d4b98f32ee2f091e7'),
  orderId: 1002,
  customer: 'Siti',
  city: 'JB',
  status: 'PENDING',
  amount: 80,
  items: [
    {
      sku: 'A1',
      qty: 1
    }
  ]
}
```

db

Refers to the **current database** in use (shopdb).

orders

The **collection name** where order records are stored.

find()

A MongoDB method used to **query documents** from a collection.

Sorting in MongoDB

Sorting allows query results to be arranged in **ascending (1)** or **descending (-1)** order.

Ascending

```
>_MONGOSH
> db.orders.find({}, { _id: 0, orderId: 1, amount: 1 }).sort({ amount: 1 })
< {
  orderId: 1004,
  amount: 50
}
{
  orderId: 1002,
  amount: 80
}
{
  orderId: 1001,
  amount: 120
}
{
  orderId: 1005,
  amount: 200
}
{
  orderId: 1003,
  amount: 300
}
>
```

It displays only the
order ID and amount
for all orders, sorted by
amount from lowest to
highest.

Descending

```
>_MONGOSH
> db.orders.find({}, { _id: 0, orderId: 1, amount: 1 }).sort({ amount: -1 })
< {
  orderId: 1003,
  amount: 300
}
{
  orderId: 1005,
  amount: 200
}
{
  orderId: 1001,
  amount: 120
}
{
  orderId: 1002,
  amount: 80
}
{
  orderId: 1004,
  amount: 50
}
>
```

It displays order ID and amount,
sorted by amount from highest
to lowest.

Sorting in MongoDB

```
> _MONGOSH
```

```
>  
>  
> db.orders.find({}, { _id: 0, orderId: 1, city: 1, amount: 1 }).sort({ city: 1, amount: -1 })
```

```
< {  
  orderId: 1002,  
  city: 'JB',  
  amount: 80  
}  
{  
  orderId: 1005,  
  city: 'KL',  
  amount: 200  
}  
{  
  orderId: 1001,  
  city: 'KL',  
  amount: 120  
}  
{  
  orderId: 1004,  
  city: 'KL',  
  amount: 50  
}  
{  
  orderId: 1003,  
  city: 'Penang',  
  amount: 300  
}
```

Sort by city then amount

It shows the order ID, city, and amount for all orders, grouped by city in alphabetical order, and within each city, sorted by amount from highest to lowest.

Advanced Query Operators: conditions, ranges, patterns, and arrays

1. Conditions : Condition operators filter documents based on **logical or comparison rules**.

Find orders with status PAID

```
>_MONGOSH
> db.orders.find({ status: "PAID" })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
},
{
  _id: ObjectId('695d535d4b98f32ee2f091e8'),
  orderId: 1003,
  customer: 'Kumar',
  city: 'Penang',
  status: 'PAID',
  amount: 300,
  items: [
    {
      sku: 'C3',
      qty: 3
    }
  ]
}
```

Combine conditions (AND)

```
>_MONGOSH
> db.orders.find({ city: "KL", status: "PAID" })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
},
{
  _id: ObjectId('695d535d4b98f32ee2f091ea'),
  orderId: 1005,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 200,
  items: [

```

Logical OR

```
>_MONGOSH
> db.orders.find({
  $or: [
    { city: "KL" },
    { city: "JB" }
  ]
})
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
},
{

```

Advanced Query Operators: conditions, **ranges**, patterns, and arrays

2. Ranges : Range operators retrieve data where values fall **within a specified range**.

Amount greater than 100

```
>_MONGOSH
> db.orders.find({ amount: { $gt: 100 } })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
}
```

Amount between 100 and 250

```
>_MONGOSH
}
> db.orders.find({ amount: { $gte: 100, $lte: 250 } })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

Advanced Query Operators: conditions, **ranges**, patterns, and arrays

Count total number of orders

```
> db.orders.aggregate([
  { $count: "totalOrders" }
])
< {
  totalOrders: 5
}
```

Count Documents by Group (GROUP COUNT)

```
> db.orders.aggregate([
  { $group: { _id: "$city", orderCount: { $sum: 1 } } }
])
< {
  _id: 'Penang',
  orderCount: 1
}
{
  _id: 'KL',
  orderCount: 3
}
{
  _id: 'JB',
  orderCount: 1
}
```

- \$group groups documents by city
- \$sum: 1 adds **1** for each document
- Result = count per city

Advanced Query Operators: conditions, ranges, **patterns**, and arrays

3. Patterns : Pattern matching allows searching **text values** using regular expressions.

Customer name starts with "A"

```
>_MONGOSH
> db.orders.find({ customer: { $regex: "^A" } })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

\$regex

- MongoDB operator for **pattern matching** using regular expressions

Case-insensitive search

```
>_MONGOSH
> db.orders.find({ customer: { $regex: "ali", $options: "i" } })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
},
{
  _id: ObjectId('695d535d4b98f32ee2f091ea'),
  orderId: 1005,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

\$options: "i"

- "**i**" means **case-insensitive**
- Uppercase and lowercase letters are treated the same

Advanced Query Operators: conditions, ranges, patterns, and arrays

4. Arrays : Array operators query **nested or array-based data** within documents.

Find orders containing SKU A1

```
>_MONGOSH
> db.orders.find({ "items.sku": "A1" })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

items.sku

- Uses **dot notation**
- items is an **array field** inside each order document.
- Tells MongoDB to **look inside each object in the items array**

Match multiple conditions in an array element

```
>_MONGOSH
}
> db.orders.find({
  items: { $elemMatch: { sku: "A1", qty: { $gte: 2 } } }
})
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

\$elemMatch

- Ensures **all conditions apply to the same array element**
- Without \$elemMatch, MongoDB may match conditions across **different elements**, which can give incorrect results

Conditions applied to **one item object**:

sku: "A1" → item SKU must be A1

qty: { \$gte: 2 } → quantity must be **2 or more**

Both conditions must be true **in the same item**.

Benefits of Indexing in MongoDB

- **Faster data retrieval**
Indexes allow MongoDB to locate required data quickly without scanning every document in a collection.
- **Improved query performance**
Queries that filter data using conditions (such as equality or range conditions) run much faster when relevant indexes are available.
- **Efficient sorting of results**
Indexes help MongoDB return sorted data directly, reducing the need for additional processing and memory usage.
- **Better scalability for large datasets**
As the size of the database grows, indexed queries continue to perform efficiently, while non-indexed queries become slower.
- **Reduced system resource usage**
By examining fewer documents, indexing reduces CPU and memory consumption on the database server.
- **Optimised aggregation operations**
Indexes improve the performance of aggregation stages that involve filtering and sorting of data.
- **Enhanced support for read-heavy applications**
Indexing is especially beneficial for applications that perform frequent searches, reporting, or monitoring.
- **Improved overall database efficiency**
Indexes help the query planner choose the most efficient execution path, leading to better overall system performance.

Indexing in MongoDB

Indexes improve query performance by allowing MongoDB to locate data faster, especially when working with large datasets.

Create Index on city

```
> db.orders.createIndex({ city: 1 })  
< city_1
```

View on all index

```
> db.orders.getIndexes()  
< [  
  { v: 2, key: { _id: 1 }, name: '_id_' },  
  { v: 2, key: { city: 1 }, name: 'city_1' },  
  { v: 2, key: { city: 1, status: 1 }, name: 'city_1_status_1' }  
]
```

Index for Filter + Sort Together

```
> db.orders.createIndex({ city: 1, amount: -1 })  
< city_1_amount_-1  
> db.orders.find({ city: "KL" })  
      .sort({ amount: -1 })  
< {  
  _id: ObjectId('695d535d4b98f32ee2f091ea'),  
  orderId: 1005,  
  customer: 'Ali',  
  city: 'KL',  
  status: 'PAID',  
  amount: 200,  
  items: [  
    {
```

Why index helps

- Filters by `city`
- Returns data **already sorted by amount**

Using Index for Range Queries in MongoDB

Create an index on the `amount` field in the `orders` collection.

Retrieve all orders where the amount is **between 100 and 250 (inclusive)**.

```
> db.orders.createIndex({ amount: 1 })
< amount_1
> db.orders.find({ amount: { $gte: 100, $lte: 250 } })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e6'),
  orderId: 1001,
  customer: 'Ali',
  city: 'KL',
  status: 'PAID',
  amount: 120,
  items: [
    {
      sku: 'A1',
      qty: 2
    }
  ],
  orderDate: 2025-12-01T00:00:00.000Z
}
```

`$gte: 100`

- Means **greater than or equal to 100**
- Includes 100

`$lte: 250`

- Means **less than or equal to 250**

Compound Index

Create a Compound Index

Create a compound index on **city** and **status**:

```
> db.orders.createIndex({ city: 1, status: 1 })  
< city_1_status_1
```

Query that Uses the Compound Index

```
> db.orders.find({ city: "KL", status: "PAID" })  
< {  
  _id: ObjectId('695d535d4b98f32ee2f091e6'),  
  orderId: 1001,  
  customer: 'Ali',  
  city: 'KL',  
  status: 'PAID',  
  amount: 120,  
  items: [  
    {  
      sku: 'A1',
```

Query Using Only the First Field (Try !)

```
> db.orders.find({ city: "KL" })  
< {  
  _id: ObjectId('695d535d4b98f32ee2f091e6'),  
  orderId: 1001,  
  customer: 'Ali',  
  city: 'KL',  
  status: 'PAID',  
  amount: 120,  
  items: [  
    {  
      sku: 'A1',
```

Query Using Only the Second Field (Try !)

Will get output however, this query does **not efficiently** use the compound index because it skips the first field.

Compound Index with Sorting

```
> db.orders.find({ city: "KL" })  
      .sort({ status: 1 })  
< {  
  _id: ObjectId('695d535d4b98f32ee2f091e6'),  
  orderId: 1001,  
  customer: 'Ali',
```

Aggregation

Aggregation pipelines process data in stages to perform **grouping, calculations, and transformations**, similar to SQL GROUP BY.

Total sales by city

```
> db.orders.aggregate([
  { $match: { status: "PAID" } },
  { $group: { _id: "$city", totalSales: { $sum: "$amount" } } }
])
< {
  _id: 'KL',
  totalSales: 320
}
{
  _id: 'Penang',
  totalSales: 300
}
```

```
SELECT city, SUM(amount) AS totalSales
FROM orders
WHERE status = 'PAID'
GROUP BY city;
```

`aggregate([ ... ])`

- `aggregate()` is used to perform **data processing and analysis**
- It works using a **pipeline**, where each stage processes the output of the previous stage

Stage 1: `$match`

```
{ $match: { status: "PAID" } }
```

What it does

- Filters documents to include **only PAID orders**
- Similar to a `WHERE` clause in SQL

Stage 2: `$group`

```
{ $group: { _id: "$city", totalSales: { $sum: "$amount" } } }
```

a) `_id: "$city"`

- Groups documents **by city**
- Each unique city becomes one output document

b) `totalSales: { $sum: "$amount" }`

- Adds up (`$sum`) the `amount` values
- Computes **total sales per city**

Aggregation – Practice !

Total spending per customer and totalSpent in descending order using MongoDB.

```
SELECT customer, SUM(amount) AS  
totalSpent  
FROM orders  
GROUP BY customer  
ORDER BY totalSpent DESC;
```

Update in MongoDB

1) Update ONE document (updateOne)

Use when you want to change only the first matching document.

Update status for orderId 1002 to PAID

```
> db.orders.updateOne(
  { orderId: 1002 },
  { $set: { status: "PAID" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.orders.find({ orderId: 1002 })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e7'),
  orderId: 1002,
  customer: 'Siti',
  city: 'JB',
  status: 'PAID',
  amount: 80,
  items: [
    {
      sku: 'A1',
      qty: 1
    }
  ],
  orderDate: 2025-12-02T00:00:00.000Z
}
```

2) Update MANY documents (updateMany)

Use when you want to change all documents that match the filter. Use when you want to change all documents that match the filter. Set status = "REVIEW" for all FAILED orders

```
> db.orders.updateMany(
  { status: "FAILED" },
  { $set: { status: "REVIEW" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.orders.find({ status: "REVIEW" })
< {
  _id: ObjectId('695d535d4b98f32ee2f091e9'),
  orderId: 1004,
  customer: 'Mei',
  city: 'KL',
  status: 'REVIEW',
  amount: 50,
  items: [
    {
      sku: 'B2',
      qty: 1
    }
  ],
  orderDate: 2025-12-03T00:00:00.000Z
}
```

Retrieve in MongoDB

Retrieve Active Login Sessions

1. Create a collection called `loginSessions` that stores user login activity.
2. Insert many sample login session records

Example :

```
userId: "U001",  
username: "ali",  
ipAddress: "10.10.1.5",  
status: "ACTIVE",  
loginTime: ISODate("2026-01-07T08:30:00Z")
```

3. Verify data

```
>_MONGOSH  
> db.loginSessions.find()  
< {  
  _id: ObjectId('695d698d2009a7259446e30f'),  
  userId: 'U001',  
  username: 'ali',  
  ipAddress: '10.10.1.5',  
  status: 'ACTIVE',  
  loginTime: 2026-01-07T08:30:00.000Z  
}  
{  
  _id: ObjectId('695d698d2009a7259446e310'),  
  userId: 'U002',  
  username: 'siti',
```

4. Retrieve all ACTIVE sessions

```
> db.loginSessions.find({ status: "ACTIVE" })  
< {  
  _id: ObjectId('695d698d2009a7259446e30f'),  
  userId: 'U001',  
  username: 'ali',  
  ipAddress: '10.10.1.5',  
  status: 'ACTIVE',  
  loginTime: 2026-01-07T08:30:00.000Z  
}  
{  
  _id: ObjectId('695d698d2009a7259446e311'),
```

Delete in MongoDB

Attempt this query !

Delete one ACTIVE session for user ali

```
db.loginSessions.deleteOne({ username: "ali", status: "ACTIVE" })
```

Delete ALL inactive sessions

```
db.loginSessions.deleteMany({ status: "INACTIVE" })
```

Delete ALL documents in a collection (use with care)

```
db.loginSessions.deleteMany({})
```

`deleteOne()` removes a single record,
`deleteMany()` removes multiple records, and
`drop()` removes the entire collection.

Drop the entire collection (It's destructive)

```
db.loginSessions.drop()
```

Deletes:

- Collection
- All documents
- All indexes

EXERCISE

Database: shopdb

Collection: orders

Question 1 – Advanced Query (Condition + OR)

Retrieve all orders where the status is either PAID or PENDING, but city is NOT KL.

Question 2 – Sorting (Multi-field)

Display only `orderId`, `city`, and `amount`, sorted by:

- city in descending order
- amount in ascending order

Question 3 – Aggregation (Count under Condition)

Calculate the number of orders per city where the amount is greater than 100.

Question 4 – Indexing (Compound Index Usage)

Create a compound index that optimises queries filtering by customer and status, then write one query that benefits from this index.

Question 5 – Data Modification (Update + Delete)

- Update all orders with status "FAILED" to "CLOSED".
- Delete all orders where the amount is less than 70.



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

***Innovating Solutions
Menginovasi Penyelesaian***